

CSC 4520/6520

Design & Analysis of Algorithms

CHAPTER 4: DECREASE-AND-CONQUER

(INSERTION SORT , TOPOLOGICAL SORTING)

Abdullah Bal, PhD

Objectives



Recap

Graph Traversal

DFS

BFS



Decrease-and- Conquer



Insertion Sort



Topological Sorting

Decrease-and-Conquer

1. Reduce problem instance to smaller instance of the same problem
2. Solve smaller instance
3. Extend solution of smaller instance to obtain solution to original instance

Decrease-and-Conquer

- Can be implemented either top-down (recursive implementation) or bottom-up (nonrecursive):
- The bottom-up variation is usually implemented iteratively, starting with a solution to the smallest instance of the problem; it is called sometimes the incremental approach.
- There are **three major variations** of decrease-and-conquer:
 - Decrease by a **constant**
 - Decrease by a **constant factor**
 - **Variable size** decrease

Types of Decrease and Conquer

Decrease by a constant (usually by 1):

- Insertion sort
- Topological sorting
- Algorithms for generating permutations, subsets

Decrease by a constant factor (usually by half)

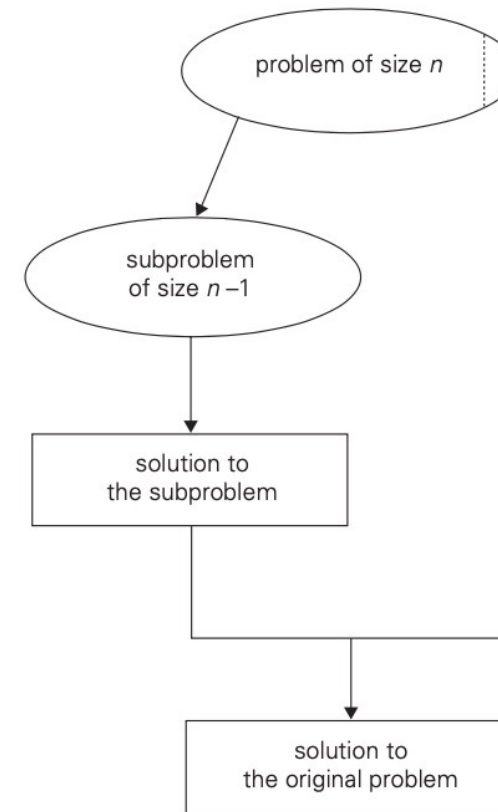
- Binary search and bisection method
- Exponentiation by squaring
- Multiplication à la russe

Variable-size decrease

- Euclid's algorithm
- Selection by partition
- Nim-like games

Decrease by a Constant

- The size of an instance is reduced by the **same constant on each iteration** of the algorithm.
- Typically, this constant is **equal to one**, although other constant size reductions do happen occasionally.

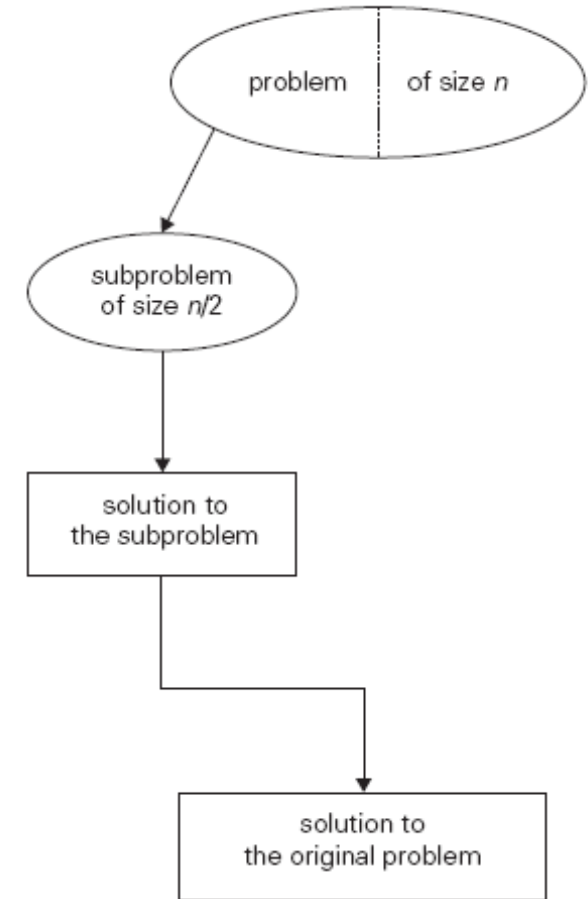


Decrease by a Constant

- The exponentiation problem of computing a^n where $a \neq 0$ and n is a nonnegative integer.
- The relationship between a solution to an instance of size n and an instance of size $n - 1$ is obtained by the obvious formula $a^n = a^{n-1} \cdot a$
- So the function $f(n) = a^n$ can be computed either “top down” by using its recursive definition or “bottom up” by multiplying 1 by a n times.

Decrease by a Constant Factor

- The ***decrease-by-a-constant-factor*** technique suggests reducing a problem instance **by the same constant factor on each iteration** of the algorithm.
- In most applications, this constant factor is equal to **two**.



Variable-size Decrease

- In the ***variable-size-decrease*** variety of decrease-and-conquer, the size-reduction **pattern varies** from one iteration of an algorithm to another.
- **Euclid's algorithm** for computing the greatest common divisor provides a good example of such a situation.

$$\text{gcd}(m, n) = \text{gcd}(n, m \bmod n).$$

- Though the value of the second argument is always smaller on the right-hand side than on the left-hand side, it decreases neither by a constant nor by a constant factor.

Insertion Sort



Insertion Sort

- To sort array $A[0..n-1]$, sort $A[0..n-2]$ recursively and then insert $A[n-1]$ in its proper place among the sorted $A[0..n-2]$
- Usually implemented bottom up (nonrecursively)
- Example: Sort 6, 4, 1, 8, 5

```
6 | 4 1 8 5
4 6 | 1 8 5
1 4 6 | 8 5
1 4 6 8 | 5
1 4 5 6 8
```

Pseudocode of Insertion Sort

ALGORITHM *InsertionSort*($A[0..n - 1]$)

//Sorts a given array by insertion sort

//Input: An array $A[0..n - 1]$ of n orderable elements

//Output: Array $A[0..n - 1]$ sorted in nondecreasing order

for $i \leftarrow 1$ **to** $n - 1$ **do**

$v \leftarrow A[i]$

$j \leftarrow i - 1$

while $j \geq 0$ **and** $A[j] > v$ **do**

$A[j + 1] \leftarrow A[j]$

$j \leftarrow j - 1$

$A[j + 1] \leftarrow v$

Analysis of Insertion Sort

- The **basic operation** of the algorithm is the key comparison $A[j] > v$.
- Why not $j \geq 0$?
- Because it is almost certainly faster than the former in an actual computer implementation.
- The number of key comparisons in this algorithm obviously depends on the nature of the input.

Analysis of Insertion Sort

- In the worst case, $A[j] > v$ is executed the largest number of times, i.e., for every
- $j = i - 1, \dots, 0$.
- For the worst-case input, we get $A[0] > A[1]$ (for $i = 1$),
 $A[1] > A[2]$ (for $i = 2$), $\dots$, $A[n - 2] > A[n - 1]$ (for $i = n - 1$).
- In other words, **the worst-case** input is an array of strictly decreasing values. **The number of key comparisons** for such an input is

$$C_{worst}(n) = \sum_{i=1}^{n-1} \sum_{j=0}^{i-1} 1 = \sum_{i=1}^{n-1} i = \frac{(n-1)n}{2} \in \Theta(n^2).$$

- In the worst case, insertion sort makes exactly the same number of comparisons as selection sort

Analysis of Insertion Sort

- In the best case, the comparison $A[j] > v$ is executed only once on every iteration of the outer loop.
- It happens if and only if $A[i - 1] \leq A[i]$ for every $i = 1, \dots, n - 1$, i.e., if the input array is already sorted in nondecreasing order.
- Thus, for sorted arrays, the number of key comparisons is

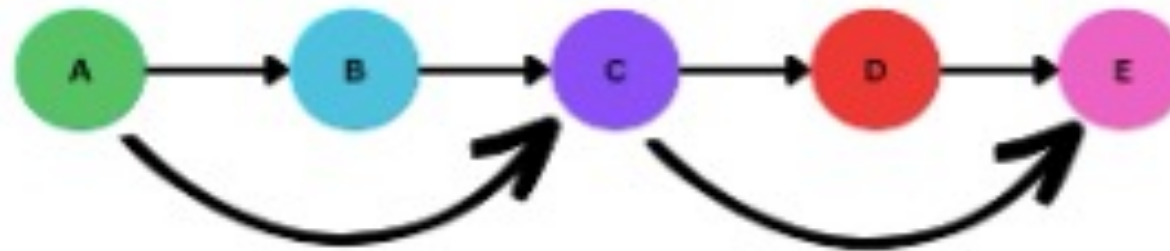
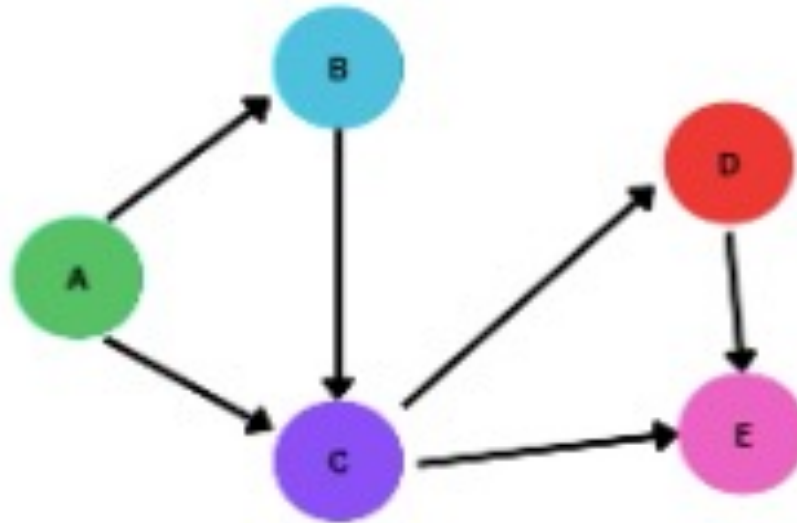
$$C_{best}(n) = \sum_{i=1}^{n-1} 1 = n - 1 \in \Theta(n).$$

Analysis of Insertion Sort

- A rigorous analysis of the algorithm's average-case efficiency is based on investigating the number of element pairs that are out of order.
- It shows that on randomly ordered arrays, insertion sort makes on **average half as many comparisons as** on decreasing arrays, i.e.,

$$C_{avg}(n) \approx \frac{n^2}{4} \in \Theta(n^2).$$

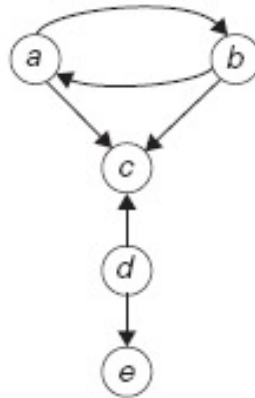
Topological Sorting



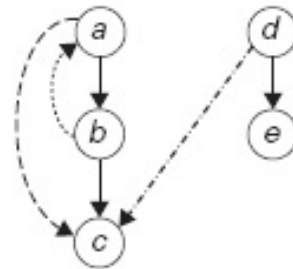
Topological Sorting

- A **directed graph**, or **digraph** for short, is a graph with directions specified for all its edges.
- There are only two notable differences between undirected and directed graphs in representing them:
 - 1) The **adjacency matrix** of a directed graph does not have to be symmetric;
 - 2) An edge in a directed graph has just one (not two) corresponding nodes in the **digraph's adjacency lists**.

○



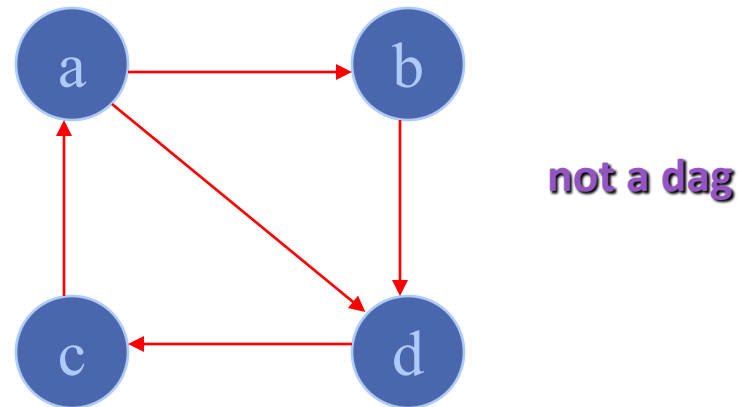
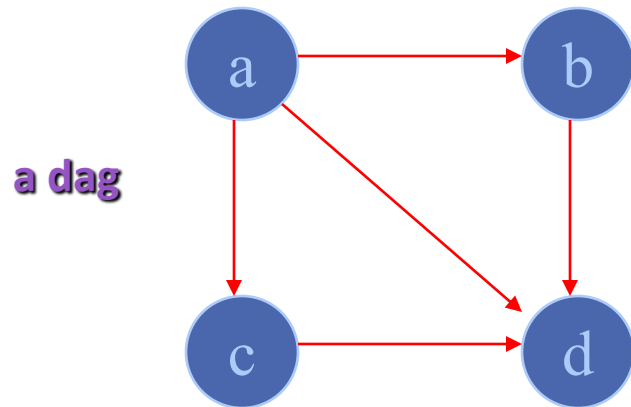
Digraph.



DFS forest of the digraph
for the DFS traversal
started at a.

Dags and Topological Sorting

- A dag: a directed acyclic graph, i.e. a directed graph with no (directed) cycles
- If a DFS forest of a digraph has no back edges, the digraph is a ***dag***, an acronym for ***directed acyclic graph***.

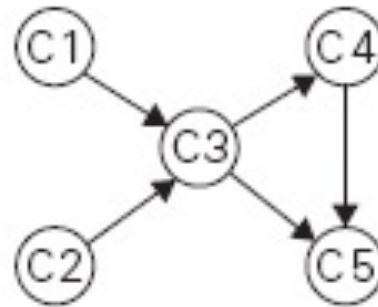


Dags and Topological Sorting

- Arise in modeling many problems that involve prerequisite constraints (construction projects, document version control, course selection etc.)
- **Topological Sorting** : Vertices of a dag can be linearly ordered so that for every edge its starting vertex is listed before its ending vertex.
- Being a dag is also a necessary condition for topological sorting be possible.

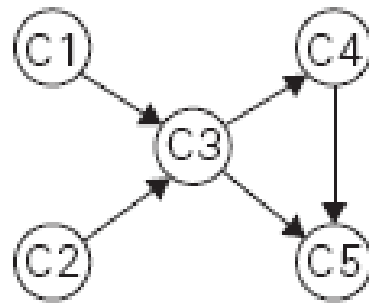
Example: Course Selection

- Consider a set of five required courses $\{C1, C2, C3, C4, C5\}$ a part-time student has to take in some degree program.
- The courses can be taken in any order as long as the following course prerequisites are met: C1 and C2 have no prerequisites, C3 requires C1 and C2, C4 requires C3, and C5 requires C3 and C4.
- The situation can be modeled by a digraph in which **vertices represent courses** and **directed edges indicate prerequisite requirements**.
- The student can take only one course per term. In which order should the student take the courses?



The First Algorithm: DFS

- In terms of this digraph, the question is whether we can list its vertices in such an order that for every edge in the graph, the vertex where the edge starts is listed before the vertex where the edge ends.
- This problem is called *topological sorting*.



Stack

C5₁
C4₂
C3₃
C1₄ C2₅

The topologically sorted list:



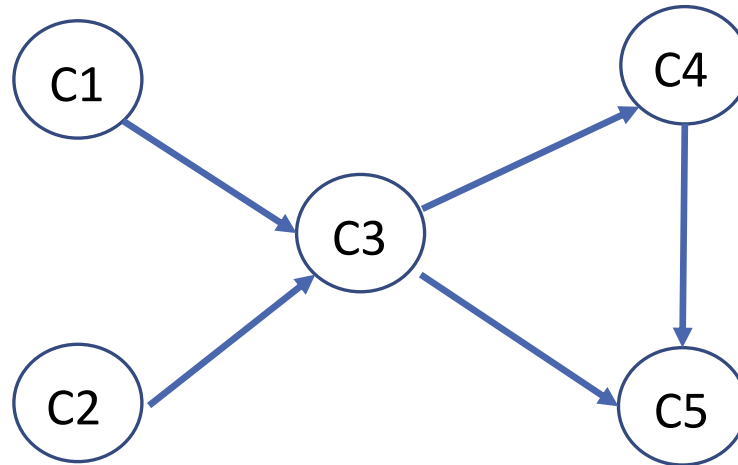
- DFS traversal stack with the subscript numbers indicating the popping- off order.

The second algorithm: Source-removal algorithm

- The second algorithm is based on a **direct implementation** of the **decrease-and-conquer technique (by one)**
- Repeatedly, identify in a remaining digraph a ***source***, which is a vertex with **no incoming edges (Indegree=0)**, and delete it along **with all the edges outgoing from it**.
- If there are several sources, **break the tie arbitrarily**.
- If there are none, stop because the problem **cannot be solved**.
- The order in which the vertices are deleted yields a solution to the topological sorting problem.

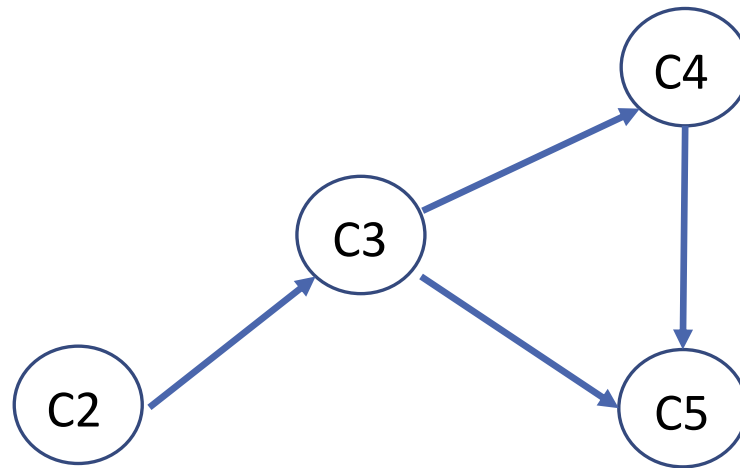
Example 1: Source-removal algorithm

- Vertices which have no incoming edges?
- Indegree=0



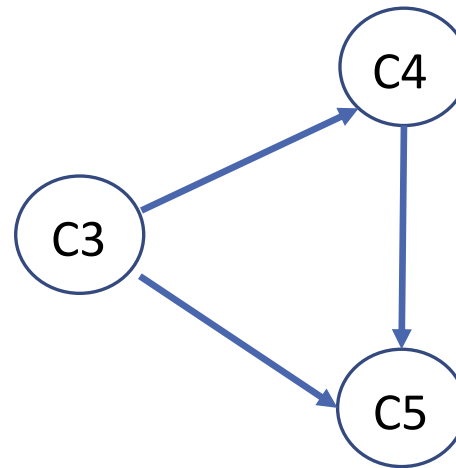
The solution: ?

Example 1: Source-removal algorithm



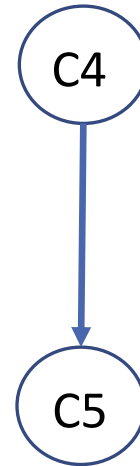
The solution: C1

Example 1: Source-removal algorithm



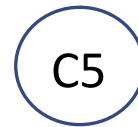
The solution: C1 C2

Example 1: Source-removal algorithm



The solution: C1 C2 C3

Example 1: Source-removal algorithm

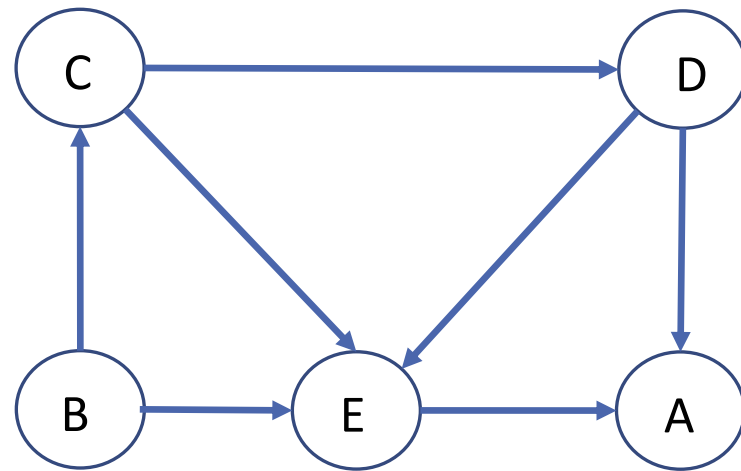


The solution: C1 C2 C3 C4

Example 1: Source-removal algorithm

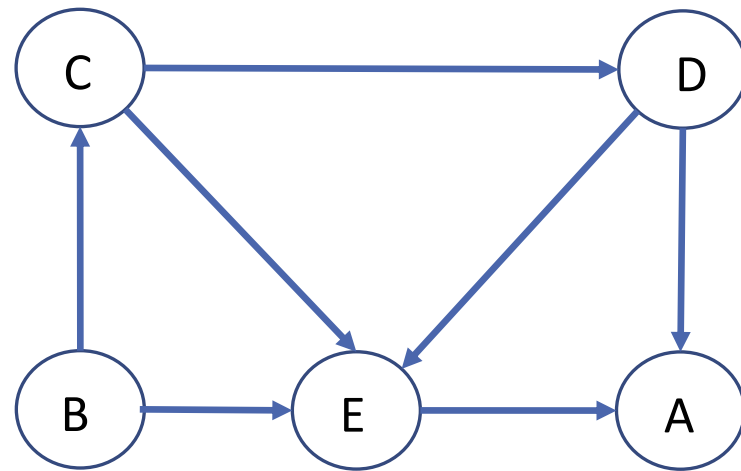
The source-removal algorithm solution: C1 C2 C3 C4 C5

Example 2: Source-removal algorithm



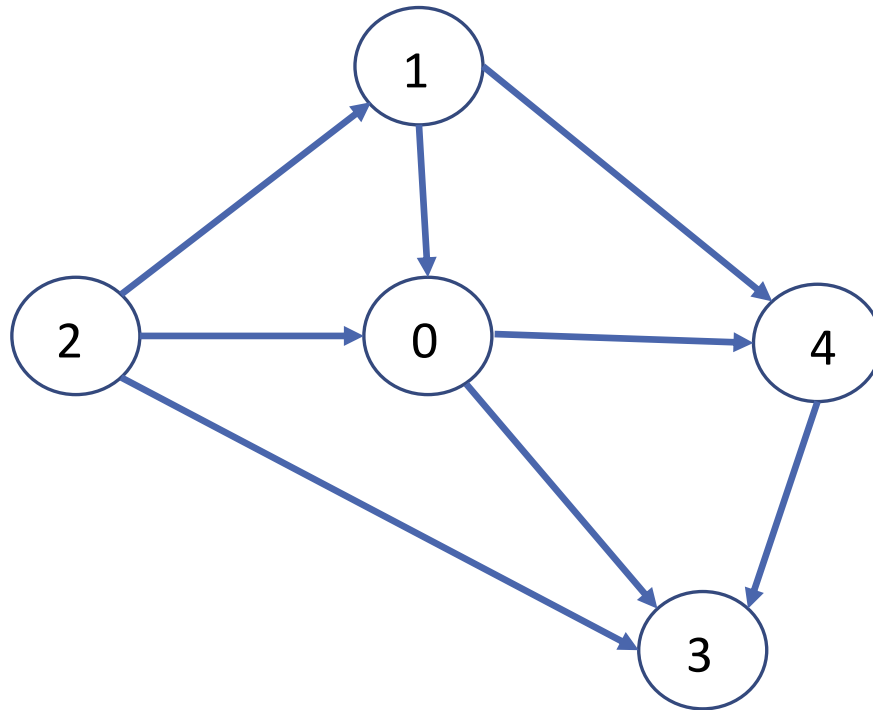
Topological sorting: ?

Example 2: Source-removal algorithm



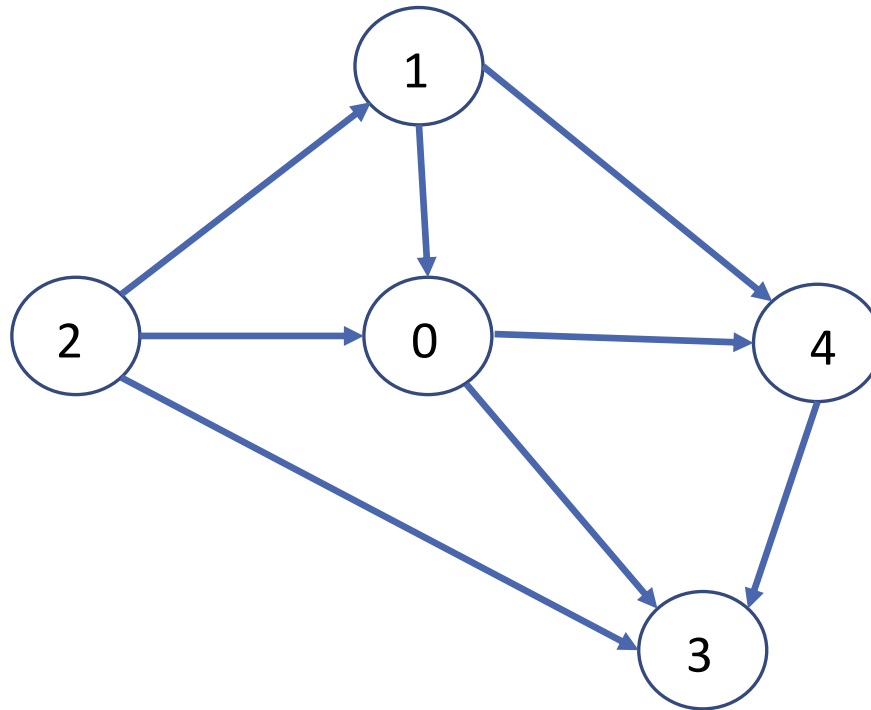
Topological sorting: B C D E A

Example 3: Source-removal algorithm



Topological sorting: ?

Example 3: Source-removal algorithm



Topological sorting: 2 1 0 4 3

DFS vs. Source-Removal Algorithm

- Note that the solution obtained by the source-removal algorithm is **different** from the one obtained by the DFS-based algorithm.
- **Both of them are correct**, of course; the topological sorting problem may have several alternative solutions.

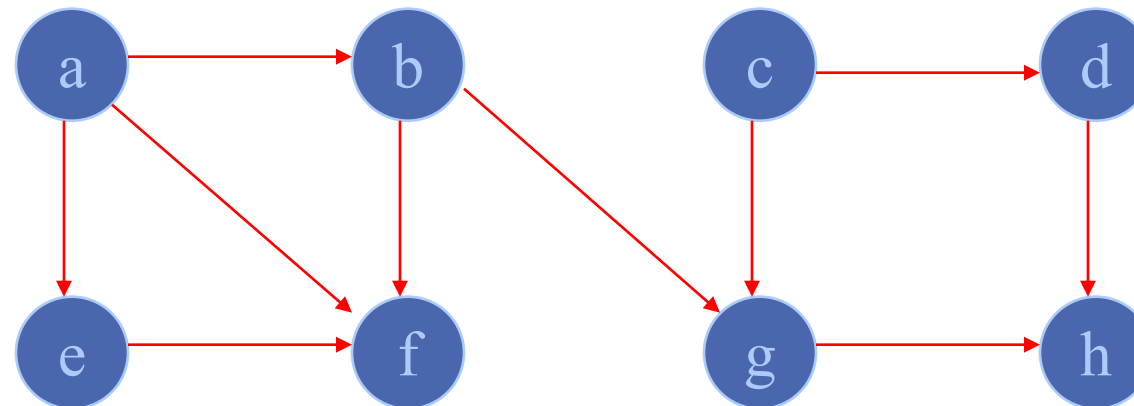
The DFS solution: C2 C1 C3 C4 C5

The source-removal algorithm solution: C1 C2 C3 C4 C5

DFS-Based Algorithm

- DFS-based algorithm for topological sorting
 - Perform DFS traversal, noting the **order vertices are popped off the traversal stack**
 - Reverse order solves topological sorting problem
 - Back edges encountered? → NOT a dag!

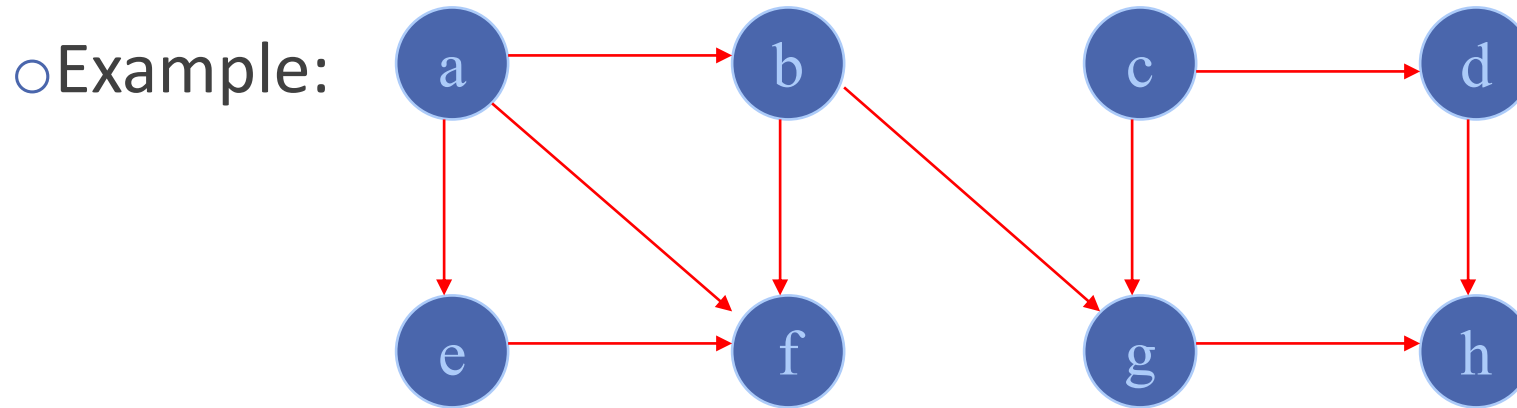
○ Example:



Source Removal Algorithm

- Source removal algorithm

- Repeatedly identify and remove a *source* (a vertex with no incoming edges) and all the edges incident to it until either no vertex is left (problem is solved) or there is no source among remaining vertices (not a dag)



- Efficiency: same as efficiency of the DFS-based algorithm

Summary

- Decrease-and-Conquer
- Insertion Sort
- Topological Sorting

Supportive Materials

- *Introduction to The Design and Analysis of Algorithms (3rd Edition)* by Anany Levitin, Pearson.
- *Introduction to Algorithms*, by T.Cormen, C. Leiserson, R.Rivest and C.Stein, MIT Press, 3rd Edition.